

planarRefiner for OpenFOAM Foundation 14

Reusable 2-D Slab and Wedge/Axisymmetric Adaptive Refinement and Automatic Unrefinement

Version 1.1.0 - September 2026

Component identity. `planarRefiner` is a runtime-selectable OpenFOAM 14 **`fvMeshTopoChanger`**, built as `libplanarFvMeshTopoChangers.so`. It is not an `fvModel` and does not depend on `detonationFoam`. Version 1.1.0 adds opt-in reversible automatic unrefinement while preserving the original refinement-only mode as the default.

Table of Contents

1. Purpose and scope.....	2
2. Why topology change belongs in <code>fvMeshTopoChanger</code>	2
2.1 <code>fvModel</code> versus <code>fvMeshTopoChanger</code>	2
3. Refinement and unrefinement theory.....	2
3.1 Cell selection.....	2
3.2 Directional planar split.....	3
3.3 Geometry detection: auto, slab, wedge.....	3
3.4 Reversible split tree and hysteretic automatic unrefinement.....	3
3.5 Per-cell cut depth and repeated cycles.....	4
3.6 Field mapping and conservation.....	4
3.7 Restart state.....	4
3.8 MPI distribution.....	4
4. Independent build.....	5
4.1 From the release root.....	5
4.2 Directly from the source directory.....	5
5. Basic <code>dynamicMeshDict</code>	5
6. Slab/empty usage.....	6
6.1 Mesh requirement.....	6
6.2 Uniform one-level smoke.....	6
6.3 Localized refinement.....	6
6.4 Qualified reversible slab cycles.....	7
7. Wedge/axisymmetric usage.....	7
7.1 Geometry.....	7
7.2 Qualified portability case.....	7
8. MPI and native load balancing.....	8
8.1 Refinement-only mode.....	8
8.2 Reversible mode.....	8
9. Restart behavior.....	8
9.1 Refinement-only restart.....	8
9.2 Reversible-mode restart.....	8
10. Using <code>planarRefiner</code> with <code>detonationFoam</code>	9
11. Using <code>planarRefiner</code> without <code>detonationFoam</code>	9
12. Limitations and safe interpretation.....	9
13. Troubleshooting.....	10
13.1 <code>planarRefiner</code> requires a 2-D mesh.....	10
13.2 Geometry mismatch.....	10
13.3 Refinement is skipped.....	10
13.4 Automatic unrefinement is skipped.....	10
13.5 Restart is refused in reversible mode.....	10
13.6 Runtime redistribution/load balancing is refused.....	11
13.7 Restart refines twice in refinement-only mode.....	11
13.8 MPI redistribution uses simple instead of Scotch.....	11
14. v1.1.0 qualification summary.....	11
15. Source layout.....	11

1. Purpose and scope

OpenFOAM 14 has strong native 3-D dynamic refinement support. The purpose of `planarRefiner` is narrower: provide a reusable dynamic-refinement path for **true two-dimensional** meshes where topology change occurs only in the two active geometric directions.

Version 1.1.0 supports:

- planar slab meshes using empty constraint patches;
- wedge/axisymmetric meshes using paired wedge patches;
- uniform or localized scalar-field-band refinement;
- optional reversible automatic unrefinement/coarsening;
- hysteresis between refinement and coarsening thresholds;
- repeated refine/unrefine cycles using per-cell binary cut depth;
- refinement-only MPI redistribution callbacks and native runtime load balancing;
- refinement-only restart from an already refined/distributed mesh;
- fixed-decomposition MPI operation in reversible mode;
- safe restart from a fully coarsened reversible state.

`automaticUnrefinement` defaults to `false`. Thus existing v1.0.0 dictionaries retain their refinement-only behavior.

When automatic unrefinement is enabled, OpenFOAM's reversible `undoableMeshCutter` ancestry is maintained only during the uninterrupted run. Version 1.1.0 therefore deliberately refuses active-history restart, mesh-to-mesh mapping, and runtime mesh redistribution/load balancing in reversible mode. These restrictions prevent invalid coarsening when the parent/sibling ancestry is unavailable.

2. Why topology change belongs in `fvMeshTopoChanger`

2.1 `fvModel` versus `fvMeshTopoChanger`

An OpenFOAM `fvModel` primarily contributes source terms to finite-volume equations. It can receive mesh-change callbacks so its own data survives topology changes, but it is not the normal owner of dynamic topology operations.

An `fvMeshTopoChanger` is the runtime-selected mesh component that participates in the standard `mesh.update()` topology-change path. Therefore `planarRefiner` is implemented as:

```
class planarRefiner : public fvMeshTopoChanger
```

and registered with:

```
addToRunTimeSelectionTable(fvMeshTopoChanger, planarRefiner, fvMesh);
```

This is why the component can be used by multiple solvers without embedding AMR code inside each solver.

3. Refinement and unrefinement theory

3.1 Cell selection: $L_r \leq \phi_c \leq U_r$

The user selects a registered `volScalarField` and a lower/upper refinement band. Cell c is selected if

$$L_r \leq \phi_c \leq U_r.$$

For a detonation case, a representative temperature band is

$$800\text{ K} \leq T_c \leq 4900\text{ K},$$

which targets the shock/reaction transition rather than the entire domain. The numerical values are case-specific and should not be treated as universal defaults.

3.2 Directional planar split: $N_{new} = N_{old} + 3 N_{sel}$

A true 2-D mesh has one inactive geometric direction. `planarRefiner` queries `mesh.geometricD()` and refines only the two active directions using OpenFOAM's directional refinement machinery.

One selected parent is split in two directions, producing four planar children. Therefore one planar level gives

$$N_{new} = N_{old} + 3 N_{selected}.$$

The qualified localized slab selected 384 cells from a 1200-cell base mesh and produced

$$1200 + 3(384) = 2352$$

cells exactly.

Internally, one planar level consists of **two binary directional cuts**. This fact is important for reversible unrefinement: two undo passes can collapse the four children back to the parent when both binary split levels are eligible.

3.3 Geometry detection: `auto`, `slab`, `wedge`

Supported settings are:

```
geometry auto;
geometry slab;
geometry wedge;
```

`auto` examines boundary patch types:

- one or more empty patches and no wedge patches -> `slab`;
- at least two wedge patches and no empty patches -> `wedge`.

Mixed/ambiguous geometry is rejected rather than guessed. The mesh must report exactly two geometric dimensions:

$$n_{geometricD} = 2.$$

3.4 Reversible split tree and hysteretic automatic unrefinement

With

```
automaticUnrefinement true;
```

`planarRefiner` creates and retains an OpenFOAM `undoableMeshCutter`. Coarsening candidates come from this reversible split tree; the code does not attempt to merge arbitrary neighboring cells heuristically.

Let the refinement band be $[L_r, U_r)$ and the wider unrefinement limits be L_u and U_u , with

$$L_u \leq L_r < U_r \leq U_u.$$

A reversible split face with sibling-cell indicator values ϕ_o and ϕ_n is eligible only if both siblings have left the active band on the **same side**:

$$(\phi_o < L_u \wedge \phi_n < L_u) \text{ or } (\phi_o > U_u \wedge \phi_n > U_u).$$

This prevents coarsening a split that still straddles a shock/reaction feature. The gap between the refinement and unrefinement thresholds provides hysteresis and reduces refine/coarsen chatter.

A practical starting point is therefore

```
lowerUnrefineLevel <= lowerRefineLevel;  
upperUnrefineLevel >= upperRefineLevel;
```

with enough separation to accommodate normal indicator fluctuations.

3.5 Per-cell cut depth and repeated cycles

The original refinement-only mode used a global successful-refinement counter. That is unsuitable for repeated reversible cycles because a cell that has been coarsened should be allowed to refine again.

In reversible mode, `maxRefinementLevel` is enforced through per-cell **binary cut depth**. Since one planar level is two binary cuts, a requested planar level corresponds internally to a maximum binary depth of

$$d_{max} = 2 L_{max},$$

where L_{max} is `maxRefinementLevel`.

This permits sequences such as

refine → unrefine → refine → unrefine

without exhausting a global event count.

3.6 Field mapping and conservation

Topology changes are applied through the normal OpenFOAM `fvMesh` update/mapping path. Registered fields are mapped with the topology map; `planarRefiner` does not keep private copies of flow/species fields.

The v1.1.0 qualification monitored mesh volume plus a uniform scalar and a nonuniform tracer through repeated refinement/unrefinement. Relative integral changes were no larger than

$$2 \times 10^{-10}.$$

This is a compact mapping/conservation gate, not a guarantee that every solver-specific flux correction is appropriate. A solver used with `planarRefiner` must still implement its normal OpenFOAM topology-change lifecycle correctly.

3.7 Restart state

The component writes:

```
<time>/polyMesh/planarRefinerState
```

In refinement-only mode this stores the global refinement-iteration count, preventing an already-refined restart from applying the same iteration again.

In reversible mode the state also records whether active reversible ancestry existed when the state was written. Because the complete `undoableMeshCutter` ancestry is not serialized, a restart from a time containing active reversible splits is intentionally refused. A restart from a **fully coarsened** state was qualified and can refine again.

3.8 MPI distribution

In refinement-only mode, `planarRefiner` participates in native OpenFOAM mesh redistribution and receives

```
planarRefiner::distribute(const polyDistributionMap&)
```

callbacks. This mode was qualified with native load balancing and refined/distributed restart.

In reversible mode, fixed-decomposition MPI2 was qualified. Runtime redistribution/load balancing is intentionally refused while reversible ancestry is active because the cutter history is not distributed.

4. Independent build

4.1 From the release root

Source OpenFOAM Foundation 14 and run:

```
./AllwmakeAMR
```

The detonation solver is not compiled by this command.

4.2 Directly from the source directory

```
cd src/planarFvMeshTopoChangers
./Allwmake
```

Both paths build:

```
$FOAM_USER_LIBBIN/libplanarFvMeshTopoChangers.so
```

The library links only against OpenFOAM mesh/topology/finite-volume libraries; it does not link against libdetonationFluidSolver.so.

5. Basic `dynamicMeshDict`

A representative reversible slab configuration is:

```
FoamFile
{
    format    ascii;
    class     dictionary;
    object    dynamicMeshDict;
}

topoChanger
{
    type                planarRefiner;
    libs                ("libplanarFvMeshTopoChangers.so");

    geometry            slab;
    refineInterval      1;
    field               T;
    lowerRefineLevel    800;
    upperRefineLevel    4900;
    maxCells            200000;
    maxRefinementIterations 1;

    automaticUnrefinement true;
    unrefineInterval    1;
    lowerUnrefineLevel  700;
    upperUnrefineLevel  5000;
    maxRefinementLevel  1;
    maxUnrefinementPassesPerUpdate 2;
}
```

Parameter meanings:

Entry	Meaning
type	must be planarRefiner
libs	loads libplanarFvMeshTopoChangers.so
geometry	auto, slab, or wedge

Entry	Meaning
<code>refineInterval</code>	attempt refinement every N time indices; must be ≥ 1
<code>field</code>	registered scalar field used for selection
<code>lowerRefineLevel</code>	inclusive lower refinement threshold
<code>upperRefineLevel</code>	inclusive upper refinement threshold
<code>maxCells</code>	global hard cap
<code>maxRefinementIterations</code>	global successful-refinement cap when automatic unrefinement is off; default 1
<code>automaticUnrefinement</code>	enable reversible automatic coarsening; default false
<code>unrefineInterval</code>	attempt coarsening every N time indices; defaults to <code>refineInterval</code>
<code>lowerUnrefineLevel</code>	low-side coarsening threshold; must be $\leq$ <code>lowerRefineLevel</code>
<code>upperUnrefineLevel</code>	high-side coarsening threshold; must be $\geq$ <code>upperRefineLevel</code>
<code>maxRefinementLevel</code>	maximum repeated planar levels in reversible mode; default follows <code>maxRefinementIterations</code>
<code>maxUnrefinementPassesPerUpdate</code>	maximum binary undo passes per update; default 2

For refinement-only behavior, omit the reversible entries or set:

```
automaticUnrefinement false;
```

Start reversible operation from the base/unrefined mesh. Version 1.1.0 cannot adopt ancestry for refinement that already exists when automatic unrefinement is first enabled.

6. Slab/empty usage

6.1 Mesh requirement

A slab case must be a true 2-D OpenFOAM mesh with the inactive direction represented by empty patches. `planarRefiner` validates this rather than accepting a thin 3-D mesh as a substitute.

6.2 Uniform one-level smoke

To refine every cell once, choose a band that contains the full field range, for example:

```
field          T;
lowerRefineLevel 0;
upperRefineLevel 10000;
maxRefinementIterations 1;
```

The F2 qualification started with 1200 cells and produced 4800 cells, with strict final `checkMesh` success.

6.3 Localized refinement

Localized directional refinement can create a small conformal transition region between coarse and refined cells. In the qualified detonation slab:

- base cells: 1200;
- selected cells: 384;
- final cells: 2352;

- transition polyhedra: 4;
- 7-face transition cells: 4;
- concave cells: 4;
- max non-orthogonality: 18.435 degrees;
- max skewness: 0.3333;
- $\max \left| \sum_i Y_i - 1 \right|: 1.045 \times 10^{-8}.$

Therefore a localized planar case should be judged by topology/geometry quality and field conservation, not by requiring every transition cell to remain a perfect hexahedron.

6.4 Qualified reversible slab cycles

The v1.1.0 runtime gate exercised two full refine/unrefine cycles rather than a single coarsening event.

Uniform slab:

1200 -> 4800 -> 1200 -> 4800 -> 1200

Localized slab:

1200 -> 2352 -> 1200 -> 2352 -> 1200

All serial topology states passed their checkMesh gates. The volume/uniform/tracer integral checks remained within relative tolerance 2e-10.

7. Wedge/axisymmetric usage

7.1 Geometry

A wedge case must contain a valid pair of wedge constraint patches. Use:

`geometry wedge;`

or `geometry auto;` when the boundary is unambiguous.

`planarRefiner` identifies the inactive geometric direction from OpenFOAM's geometric-dimension information and refines the two active directions only. The normal OpenFOAM topology-change correction preserves the wedge constraint.

7.2 Qualified portability case

The F3 portability test used stock OpenFOAM 14 `incompressibleFluid` through `foamRun`:

- annular wedge geometry;
- 1600 initial cells;
- one uniform planar level;
- 6400 final cells;
- both wedge patches preserved;
- strict final checkMesh: Mesh OK..

Version 1.1.0 additionally exercised the reversible wedge cycle:

1600 -> 6400 -> 1600 -> 6400 -> 1600

with the same compact conservation tolerance used for the slab gates. These tests demonstrate that `planarRefiner` is not detonation-specific and that reversible coarsening is not limited to slab geometry.

8. MPI and native load balancing

8.1 Refinement-only mode

With `automaticUnrefinement false`, `planarRefiner` can coexist with OpenFOAM's native runtime load balancer:

```
distributor
{
    type                loadBalancer;
    libs                ("libfvMeshDistributors.so");
    redistributionInterval 2;
    maxImbalance        0;
    multiConstraint      false;
}
```

The load balancer reads the active `system/decomposeParDict`. For Scotch:

```
numberOfSubdomains 2;
method scotch;
```

A genuinely exercised run should show messages such as:

```
Selecting fvMeshDistributor loadBalancer
Selecting distributor scotch
Imbalance of load ...
Redistributing mesh
planarRefiner: received native mesh-distribution callback
```

8.2 Reversible mode

With `automaticUnrefinement true`, fixed decomposition is supported and MPI2 was qualified. **Runtime redistribution/load balancing is refused** while reversible split ancestry is active. This is an intentional correctness restriction: the current component does not distribute the `undoableMeshCutter` split tree.

Do not combine a runtime `loadBalancer` with active reversible mode in v1.1.0. Use a fixed decomposition, or disable automatic unrefinement when runtime redistribution is required.

9. Restart behavior

9.1 Refinement-only restart

The original F3 restart qualification applies to `automaticUnrefinement false`. The two-rank case:

1. started from a deterministic decomposition;
2. applied localized planar refinement;
3. performed native load balancing;
4. wrote the refined/distributed mesh and `planarRefinerState`;
5. restarted in place with `startFrom latestTime`;
6. restored refinement iteration 1/1;
7. completed without a second refinement.

The continuous and restarted final geometries agreed to numerical roundoff, and the major-species NRMSE was below 10^{-9} in the qualification case.

9.2 Reversible-mode restart

The current reversible cutter ancestry is not serialized. Therefore:

- restart from a time with active reversible splits is intentionally rejected;

- restart from a fully coarsened state is allowed;
- the v1.1.0 gate restarted from the fully coarsened state and successfully refined again.

This distinction is important: the restriction is not "no restart ever"; it is "no restart while ancestry needed for a future undo is missing."

10. Using `planarRefiner` with `detonationFoam`

Build both components:

```
./Allwmake
./AllwmakeAMR
```

Then add the `topoChanger` block to `constant/dynamicMeshDict`. No changes to `applications/modules/detonationFluid` are required.

A representative detonation refinement band is:

```
field          T;
lowerRefineLevel 800;
upperRefineLevel 4900;
```

These thresholds are examples only. For reversible operation choose a wider coarsening band and start from the base mesh, for example:

```
automaticUnrefinement      true;
unrefineInterval           1;
lowerUnrefineLevel          700;
upperUnrefineLevel          5000;
maxRefinementLevel          1;
maxUnrefinementPassesPerUpdate 2;
```

The release includes a 120-cell coupled H2/O2 smoke:

```
cd tutorials/H2_O2_laptop_autoUnref_Soret_OF14
./Allrun
```

The accepted run produced repeated refinement and automatic unrefinement, reached $T_{\max} = 3581.900841$ K, and advanced the leading shock to 0.00051 m without a fatal error/FPE. The case is intentionally a software-integration smoke rather than formal detonation validation.

11. Using `planarRefiner` without `detonationFoam`

Only the AMR library must be built:

```
./AllwmakeAMR
```

A different solver can then load the library through `constant/dynamicMeshDict`. The solver must participate correctly in OpenFOAM's normal dynamic-mesh/topology-change lifecycle and must have any topology-change flux-correction settings it requires.

For example, the F3 incompressibleFluid wedge portability test required a `pcorr` solver entry and `correctPhi` yes because stock incompressible dynamic-mesh flux correction uses `pcorr` after topology change. That requirement belongs to `incompressibleFluid`, not to `planarRefiner` itself.

12. Limitations and safe interpretation

Current v1.1.0 limitations are:

- OpenFOAM Foundation 14 only;
- true 2-D meshes only (`nGeometricD()==2`);
- supported geometry is slab/empty or wedge/axisymmetric;
- scalar-field band selection only;
- active reversible-history restart is not supported;
- mesh-to-mesh mapping is refused in reversible mode;
- runtime redistribution/load balancing is refused in reversible mode;
- reversible mode must begin from the base/unrefined mesh;
- localized transition cells can be non-hex polyhedra and should be evaluated with `checkMesh` plus conservation diagnostics;
- long endurance and formal large-scale parallel testing remain deferred.

These restrictions do not apply equally to both modes. Refinement-only mode retains the previously qualified native redistribution and refined/distributed restart behavior. Fixed-decomposition MPI2 was qualified for reversible mode.

13. Troubleshooting

13.1 `planarRefiner` requires a 2-D mesh

The mesh is being recognized as three-dimensional. Check `empty/wedge` constraint patches and mesh construction. A one-cell-thick 3-D mesh is not automatically a true 2-D mesh.

13.2 Geometry mismatch

If `geometry slab`; is requested but the boundary contains `wedge` patches, or vice versa, the component exits intentionally. Use the correct constraint patches or `geometry auto`; for an unambiguous mesh.

13.3 Refinement is skipped

Possible reasons include:

- the current time index is not a `refineInterval` multiple;
- in refinement-only mode, `maxRefinementIterations` has already been reached;
- in reversible mode, the selected cells are already at `maxRefinementLevel`;
- no cells are inside the selected scalar band;
- the projected growth would exceed `maxCells`.

13.4 Automatic unrefinement is skipped

Check that:

- `automaticUnrefinement true`; is enabled;
- the current time index is an `unrefineInterval` multiple;
- the split was created during the same uninterrupted reversible run;
- both siblings are below `lowerUnrefineLevel`, or both are above `upperUnrefineLevel`.

A split whose siblings are on different sides of the hysteresis band is intentionally retained.

13.5 Restart is refused in reversible mode

If the state file records active reversible ancestry, the refusal is intentional. Restart from a fully coarsened state or rerun from the base mesh. The current component cannot reconstruct the `undoableMeshCutter` tree from a refined mesh alone.

13.6 Runtime redistribution/load balancing is refused

This is intentional with `automaticUnrefinement true`. Use fixed decomposition, or disable automatic unrefinement before using runtime load balancing.

13.7 Restart refines twice in refinement-only mode

Check that the restart time contains:

```
polyMesh/planarRefinerState
```

and that the case is restarting from the intended written time. The log should report the restored refinement iteration.

13.8 MPI redistribution uses `simple` instead of `Scotch`

This applies to refinement-only mode with runtime load balancing. The active system/decomposeParDict still says `method simple`; . Change it to `method scotch`; before runtime load-balancer construction.

14. v1.1.0 qualification summary

The reversible feature gate passed:

Case	Cell cycle	Result
uniform slab	1200 -> 4800 -> 1200 -> 4800 -> 1200	PASS
localized slab	1200 -> 2352 -> 1200 -> 2352 -> 1200	PASS
annular wedge	1600 -> 6400 -> 1600 -> 6400 -> 1600	PASS
MPI2 fixed decomposition	1200 -> 4800 -> 1200 -> 4800 -> 1200	PASS

Across the monitored cycles, relative volume/uniform/tracer integral changes were $\leq 2e-10$. Active-history restart refusal and fully-coarsened restart/re-refinement both behaved as designed.

The coupled 120-cell H2/O2 detonationFluid smoke also passed with repeated refinement and automatic unrefinement in a reacting run.

15. Source layout

```
src/planarFvMeshTopoChangers/  
  Allwmake  
  Make/files  
  Make/options  
  README.md  
  planarRefiner/  
    planarRefiner.H  
    planarRefiner.C  
    planarMultiDirRefinement.H  
    planarMultiDirRefinement.C  
    planarRefinementIterator.H  
    planarRefinementIterator.C
```

The directional refinement implementation is based on OpenFOAM Foundation topology-change machinery and remains licensed under the GPL terms carried by the source files and release LICENSE.